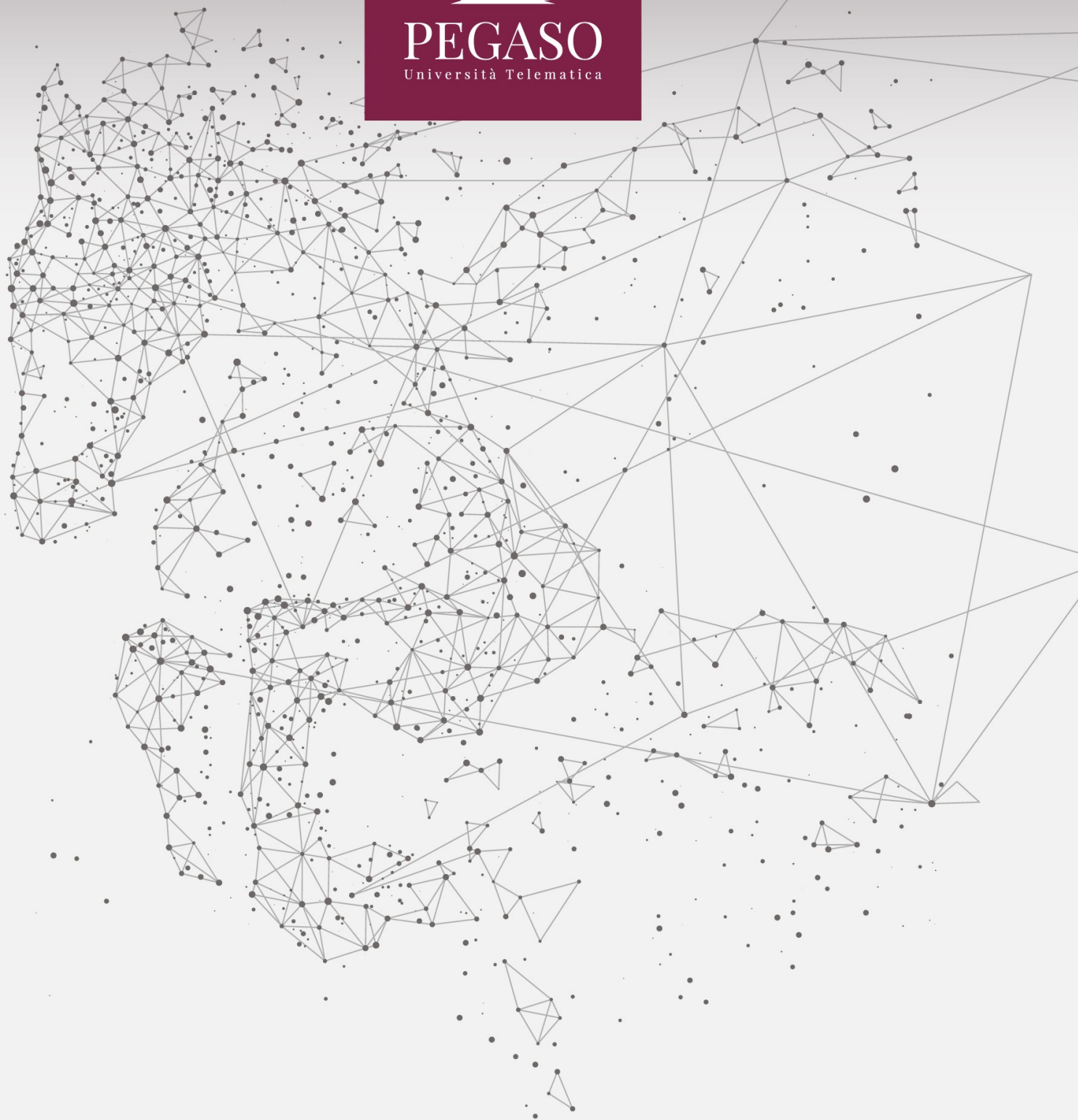




PEGASO
Università Telematica



Indice

1. PROBLEMA E SOLUZIONE	3
2. COS'È UN ALGORITMO	4
3. PROPRIETÀ DI UN ALGORITMO	6
4. SCHEMI DI COMPOSIZIONE DI UN ALGORITMO	7
5. TEOREMA DI BOHM-JACOPINI.....	10
BIBLIOGRAFIA	11

1. Problema e soluzione

Cos'è un problema? In ambito psicologico, possiamo parlare di problema ogni qual volta ci troviamo di fronte ad una discrepanza tra il nostro stato attuale e ciò che desideriamo: in questo caso, definire il problema rappresenta il primo passo per poterlo risolvere.

Per definirlo è utile porsi alcune domande la cui risposta ci consente di avere una visione più ampia ed oggettiva:

- interrogarsi circa la natura del problema;
- chiederci se disponiamo di tutte le informazioni necessarie per risolverlo;
- chiarire quali sono gli obiettivi che vogliamo raggiungere e quali soluzioni alternative abbiamo;
- valutare se coinvolgere qualcuno nella soluzione del problema;
- capire se abbiamo scelto la soluzione migliore e se sappiamo gestire le conseguenze delle nostre azioni.

Definire il problema ci consente di riflettere maggiormente e meglio su tale evento, aiutandoci a trovare soluzioni più efficaci non dettate da meccanismi automatici o emotive.

In generale, un problema è un qualcosa che siamo chiamati a risolvere. Occorre tener presente che non tutti i problemi sono però risolvibili: il fatto di non avere una soluzione ad un problema non significa che il problema non sia interessante, anzi, in qualche caso i problemi più interessanti potrebbero anche essere quelli che non hanno ancora soluzione

Da un punto di vista lessicale parliamo di:

- istanza: un particolare l'input ad un problema;
- soluzione: l'output che vi corrisponde.

Cos'è dunque un algoritmo?

2. Cos'è un algoritmo

Un algoritmo è una sequenza di comandi (dette istruzioni) che istruiscono sull'esecuzione di un determinato compito; un algoritmo è dunque una sequenza di istruzioni perfettamente comprensibili ed eseguibili tali che, se eseguite in un ordine specificato e determinato, permettono la soluzione di un problema in un numero finito di passi.

Esempi di algoritmi possono essere: la lista degli ingredienti di una ricetta di cucina e l'esatto ordine di esecuzione, uno spartito musicale, le istruzioni per il montaggio di un mobile, le istruzioni di una LEGO o, in ambito squisitamente informatico, un programma per il calcolatore.

In generale l'algoritmo è scritto da un esperto ed eseguito da un esecutore: esecutore ed esperto sono solitamente (anche se non necessariamente) due soggetti distinti; inoltre, l'algoritmo viene eseguito per un utente, che a sua volta non coincide sempre con l'esperto e l'esecutore. Gli attori, dunque, che entrano nel processo sono “almeno” 3:

- **utente generico** che esegue l'algoritmo;
- **esperto** che realizza l'algoritmo;
- **esecutore** che esegue l'algoritmo.

Esperto ed esecutore devono condividere la conoscenza del linguaggio col quale è stato scritto l'algoritmo. Tale linguaggio può essere una lingua naturale (come per esempio l'italiano, in una ricetta), una notazione particolare (es. la notazione musicale negli spartiti), un linguaggio pittografico ad immagini (come nelle figure delle istruzioni di montaggio di certi mobili) o un linguaggio artificiale di programmazione (nel caso dei programmi per un computer).

L'utente può anche ignorare completamente il linguaggio col quale è stato scritto l'algoritmo (per esempio un ascoltatore di musica può benissimo non conoscere la notazione musicale), per questo motivo parliamo di utente “generico” e lo distinguiamo dall'esecutore specifico.

I linguaggi naturali (come, per esempio, l'italiano) sono spesso ambigui, cioè possono essere interpretati in modi diversi. Immaginiamo una ricetta e supponiamo che all'interno della ricetta sia indicato: aggiungere “un pizzico di sale”. Tale istruzione è ambigua in quanto non ci dà indicazioni sulla quantità esatta di sale da aggiungere e pertanto utenti differenti potranno intendere “il pizzico di sale” in maniera differente.

Ambiguità e imprecisione non creano generalmente problemi nella comunicazione fra esseri umani dotati di intelligenza, ma non sono possibili dovendo comunicare con un computer (cioè una macchina). Per

questa ragione i linguaggi di programmazione devono consentire di scrivere algoritmi in modo chiaro, senza imprecisioni e senza nessuna possibile ambiguità.

Nel corso degli anni sono stati sviluppati molti linguaggi di scrittura algoritmi, fra i quali i più noti ed usati sono:

- linguaggio dei diagrammi di flusso (flowchart);
- pseudocodice.
- Quando parliamo di linguaggi, occorre in ogni caso tener presente 3 aspetti:
- **sintassi**: come le frasi si costruiscono;
- **semantica**: che cosa significa una frase;
- **pragmatica**: lo studio del miglior modo per esprimere lo stesso concetto.

Nel momento in cui costruiamo o analizziamo un algoritmo occorre tener presente tutti questi aspetti: il rispetto delle regole con cui è scritto (sintassi), il significato delle operazioni da eseguire (semantica), l'analisi delle soluzioni in termini di efficienza e prestazioni (pragmatica).

Alcune volte tendiamo a confondere il concetto di algoritmo con il concetto di programma: questi due non sono la stessa cosa; gli algoritmi non sono programmi. I programmi sono un modo di descrivere gli algoritmi, ed in alcuni casi lo sono in modo poco efficiente perché in alcune circostanze, il linguaggio nasconde sia l'intuizione sia di idea che soggiace all'algoritmo stesso.

A tal proposito è bene ricordare la tesi di Church-Turing:

“se un problema è umanamente calcolabile, allora esisterà una macchina di Turing in grado di risolverlo (cioè di calcolarlo)”.

Una formulazione alternativa nel nostro contesto di analisi può essere: tutti i linguaggi sufficientemente espressivi sono ugualmente espressivi, vale a dire: possiamo scegliere un linguaggio qualunque, in particolare possiamo utilizzare uno pseudocodice che il possibile ai linguaggi classici (e dunque svincolarci dallo specifico linguaggio di programmazione).

L'utilizzo di uno pseudocodice o di un flow-chart ci consente dunque di focalizzarci sulla nostra capacità di sviluppare una “intuizione algoritmica”, cioè trovare possibili soluzioni a problemi, ed in particolare, soluzioni “ottimali”. Analizzare un insieme di problemi ed un insieme di soluzioni buone (ed in alcuni casi ottime) sono importanti per poter costruire un vocabolario mentale per il nostro futuro come informatici.

3. Proprietà di un algoritmo

Analizziamo dunque le proprietà che deve possedere un algoritmo per poter essere eseguito automaticamente da una macchina:

- **finito**: la sequenza di istruzioni deve essere finita e portare ad un risultato avere un punto di inizio, dove si avvia l'esecuzione delle azioni, e un punto di fine, dove si interrompe l'esecuzione;
- **eseguibile**: le istruzioni devono poter essere eseguite materialmente dall'esecutore;
- **non ambiguo**: le istruzioni devono essere espresse in modo tale da essere interpretate da tutti allo stesso modo;
- **generale**: deve essere valido non solo per un particolare problema, ma per una classe di problemi;
- **deterministico**: partendo dagli stessi dati iniziali deve portare sempre allo stesso risultato finale indipendentemente dall'esecutore;
- **completo**: deve contemplare tutti i casi che si possono verificare durante l'esecuzione.

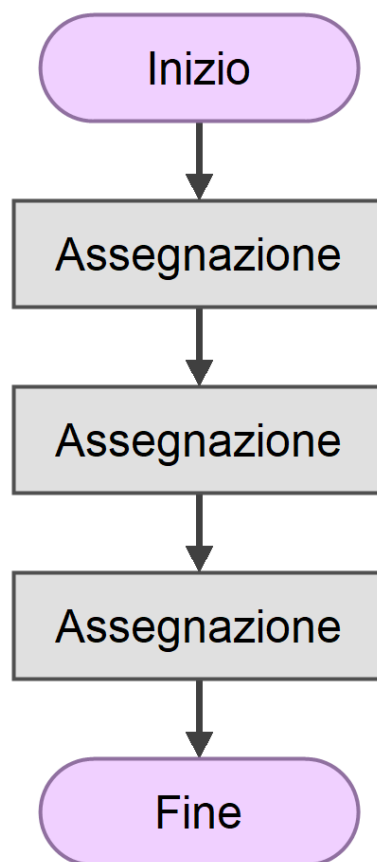
4. Schemi di composizione di un algoritmo

Possiamo in generale suddividere le istruzioni di un algoritmo in 2 categorie principali:

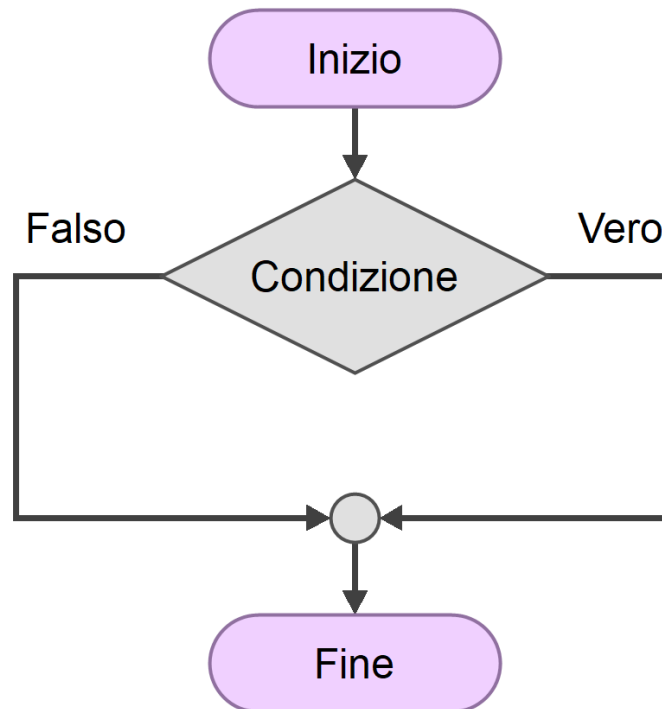
- **input:** serve all'utente per fornire un valore all'esecutore (per esempio un numero da addizionare);
- **output:** serve all'esecutore per fornire un valore all'utente (per esempio visualizzare un valore sullo schermo del computer).

Gli schemi di composizione di un algoritmo si suddividono in tre tipi:

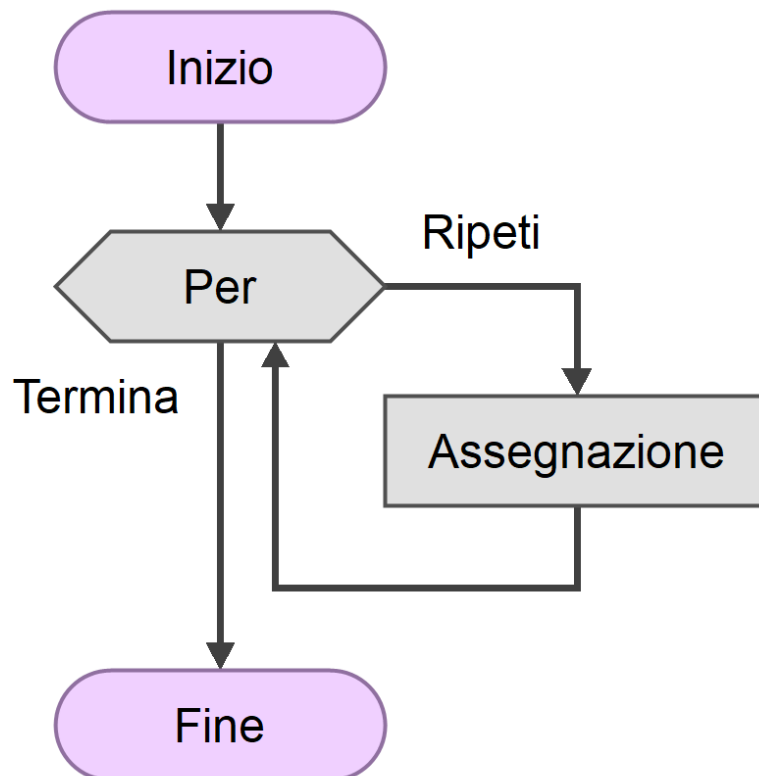
- **Sequenza:** le istruzioni sono eseguite secondo l'ordine in cui sono scritte.



- **Selezione o Condizione:** esiste una condizione da valutare e due possibili gruppi di istruzioni da eseguire; in funzione del valore della condizione, si sceglie un blocco oppure l'altro.



- **Iterazione:** consente la ripetizione di un blocco di istruzioni, per un certo numero di volte, definito a seconda dell'obiettivo da raggiungere.



Vi è inoltre un'altra categoria fondamentale di istruzione: l'assegnazione.

Questa ha a che fare con l'utilizzo delle variabili. Le variabili negli algoritmi sono simili alle variabili usate in matematica, sono cioè dei nomi usati per indicare dei valori numerici. L'uso delle variabili semplifica la scrittura degli algoritmi, ma, soprattutto, la rende di validità generale. In altre parole: un algoritmo scritto usando le variabili (al posto dei valori numerici) può essere usato per qualsiasi valore numerico; viceversa, se i valori numerici fossero indicati esplicitamente dentro l'algoritmo (in questo caso si parla di costanti, per distinguerle dalle variabili), l'algoritmo sarebbe valido per un unico caso.

L'assegnazione è dunque quell'istruzione che è utilizzata per modificare il contenuto di una variabile.

5. Teorema di Bohm-Jacopini

Il teorema di Böhm-Jacopini, enunciato nel 1966 da due informatici italiani dai quale prende il nome, afferma che:

“qualunque algoritmo può essere implementato utilizzando tre sole strutture, la sequenza, la selezione e il ciclo, da applicare ricorsivamente alla composizione di istruzioni elementari “

In altre parole, ciascun algoritmo può essere realizzato utilizzando soltanto le tre strutture indicate: sequenza, selezione e ciclo.

Da notare come tale teorema non entra nel merito della “bontà” della soluzione scelta, vale a dire che non si sta discutendo del fatto che la soluzione scelta sia la migliore o meno: si sta semplicemente dicendo che “qualora” si voglia trovare la soluzione ad un problema, questa può essere ottenuta semplicemente usando le 3 strutture di sequenza, selezione e ciclo; il problema della soluzione “migliore” è pertanto un ulteriore aspetto che è importante analizzare ma che non rientra nella specificità del teorema di Bohm-Jacopini.

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione.
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione.
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley.
- Sedgewick: Algoritmi in C, Pearson, 2015.